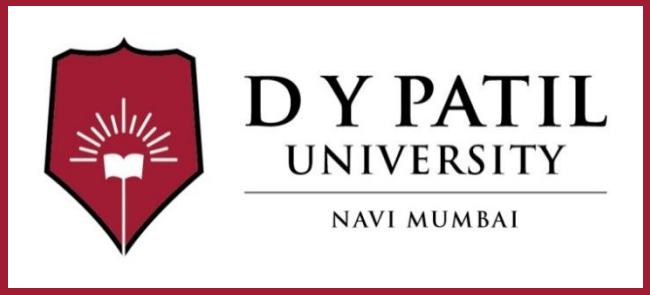
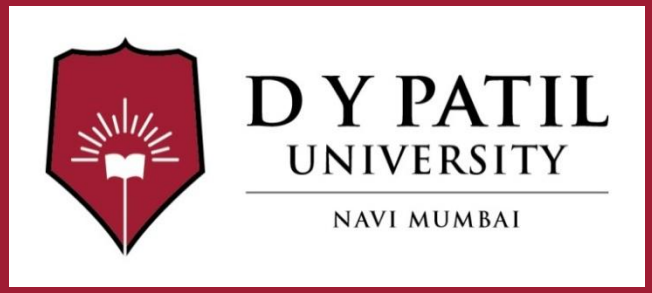


Subject Name :Web Development with Java

Unit No:IV

Unit Name :Spring Boot and RESTful Web Services

Faculty Name :Dr. Saloni Bhushan



Unit No.4

Faculty Name : Dr. Saloni Bhushan

Index

Lecture No: 1

Spring Framework and Dependency Injection

What is Spring Framework?

- Spring Framework was created by Rod Johnson(2003) and released under Apache 2.0 license.
- The most popular application development framework for enterprise Java
- An Open source Java platform
- Provides to create high performing, easily testable and reusable code.
- is organized in a modular fashion
- simplifies java development

Intro to Spring

Spring Framework

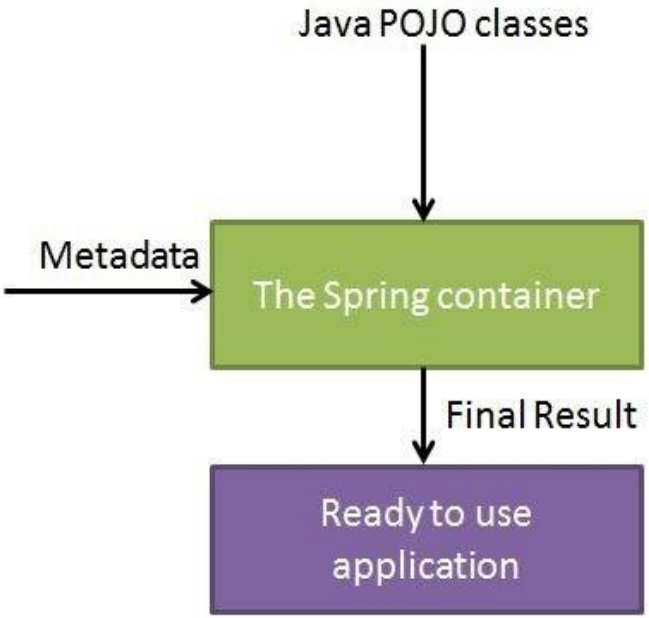
- enables Plain Old Java Object (POJO) based programming model
- is a well-designed web model-view-controller (MVC) framework
- The Inversion of Control (IoC) containers are lightweight. Being lightweight is beneficial for developing and deploying applications on computers with limited resources (RAM&CPU).
- Testing is simple because environment-dependent code is moved into this framework.

What are Beans?

- In Spring, POJO's (plain old java object) are called 'beans' and those objects instantiated, managed, created by Spring IoC container.
- Beans are created with the configuration metadata (XML file) that we supply to the container.
- Bean definition contains configuration metadata.With this information, container knows how to create bean, beans lifecycle, beans dependencies
- After specifying objects of an application, instances of those objects will be reached by getBean() method.
- Spring supports various scope types for beans:

  -default scope of a bean is Singleton (a single instance per Spring IoC container)

Big Picture of Spring (High-level view)



The Spring IoC container makes use of Java POJO classes and configuration metadata to produce a fully configured and executable system or application.

Two Key Components of Spring

- **Dependency Injection (DI)** helps you decouple your ***application objects*** from each other

- **IOC(Inversion of Control**
  Inversion of Control is a design principle where the control of object creation, configuration, and management is transferred from the application code to a container or framework. The Spring IoC container is the core component responsible for this process.

Dependency Injection (DI)

- Spring is most identified with **Dependency Injection (DI)** technology.
- DI is only one concrete example of Inversion of Control.
- In a complex Java application, classes should be loosely coupled. This feature provides code reuse and independently testing classes.
- DI helps in gluing loosely coupled classes together and at the same time keeping them independent.
- DI is preferable because it makes testing easier

Dependency Injection Types

- DI will be accomplished by given two ways:
  - *passing parameters* to the constructor (***used for mandatory dependencies***) or
  - using *setter methods(**used for optional depenedencies**)*.

Constructor Injection (for **mandatory dependencies**)

In this approach, dependencies are passed as parameters to the class constructor.

```
class Engine {
    void start() {
        System.out.println("Engine started");
    }
}
class Car {
    private Engine engine;

    // Constructor Injection
    public Car(Engine engine) {
        this.engine = engine;
    }

    void drive() {
        engine.start();
        System.out.println("Car is driving");
    }
}
```

```
Engine engine = new Engine();
Car car = new Car(engine);
// must provide engine
car.drive()
```

Here, `Engine` is **mandatory**. A `Car` cannot exist without it.

Setter Injection (for **optional dependencies**)

In this approach, dependencies are set using setter methods after the object is created.

```
class MusicSystem {
    void playMusic() {
        System.out.println("Playing music");
    }
}
class Car {
    private MusicSystem musicSystem;

    // Setter Injection
    public void setMusicSystem(MusicSystem musicSystem) {
        this.musicSystem = musicSystem;
    }
    void drive() {
        System.out.println("Car is driving");
        if (musicSystem != null) {
            musicSystem.playMusic();
        }
    }
}
```

```
Car car = new Car(); // no dependency required initially

MusicSystem music = new MusicSystem();
car.setMusicSystem(music); // optional

car.drive();
```

Here, `MusicSystem` is **optional**. The car can still work without it.

IOC Container

- The Spring container(IOC container) is at the core of the Spring Framework.
- The container will create the objects, wire them together, configure them and manage their complete lifecycle from creating till destruction.
- The container gets its instruction on what objects to instantiate, configure and assemble by reading configuration metadata provided.
- The configuration metadata can be represented either by
  - XML
  - Java annotations
  - Java Code
-

IOC is achieved through Dependency Injection

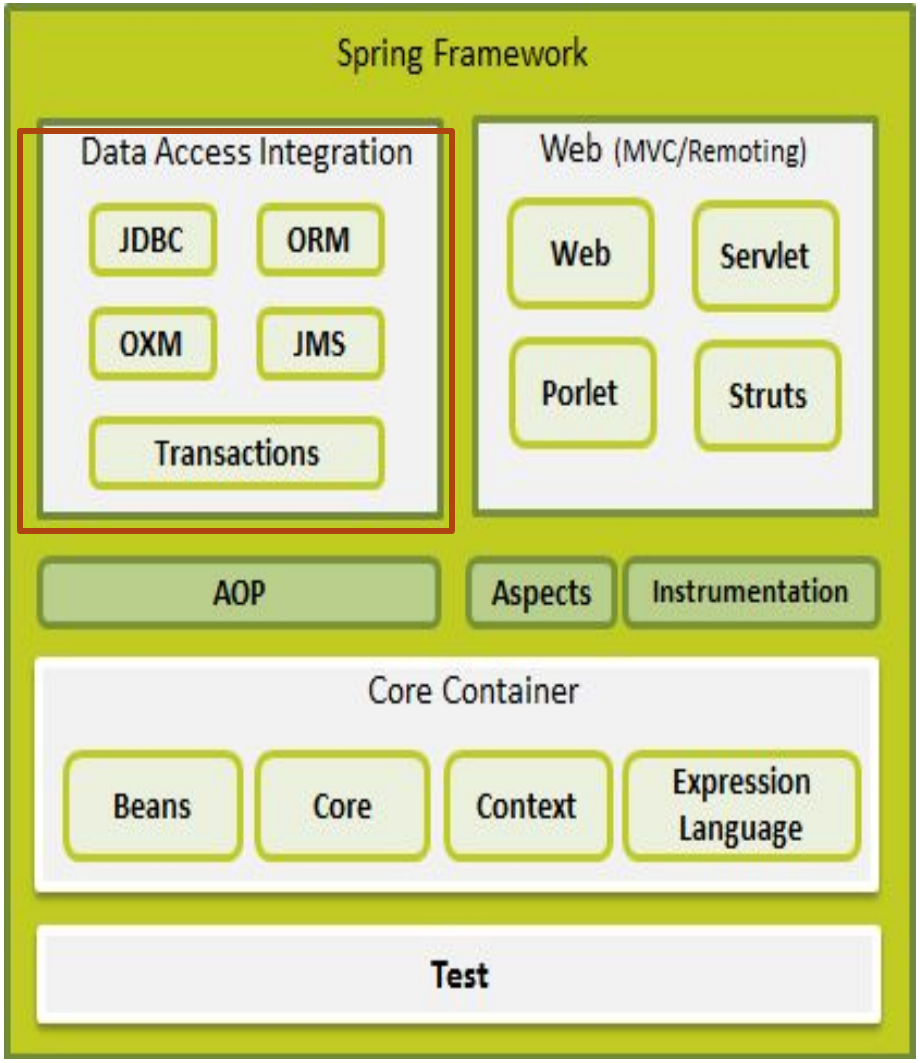
IoC Containers:

- **BeanFactory:** The basic, lightweight container providing fundamental DI features.
- **ApplicationContext:** The advanced container used in most enterprise applications. It extends `BeanFactory` with additional features like event handling, AOP integration, and internationalization.

Spring Framework Architecture

Data Access Layer
Simplyfies data access with JDBC, ORM, OXM, JMS and transaction Management

Web MVC/Remoting
Supports building web applications using spring MVC, Servlet, Portlets and struts.



Spring Framework Architecture- Web MVC Layer

A **portlet** is a Java-based component that processes requests and produces dynamic content, which is then combined with other portlets to form a complete portal page.

Imagine a dashboard page like this:

- Weather widget
- News feed

Each of these sections is a **portlet**, and the whole page is called a **portal**.

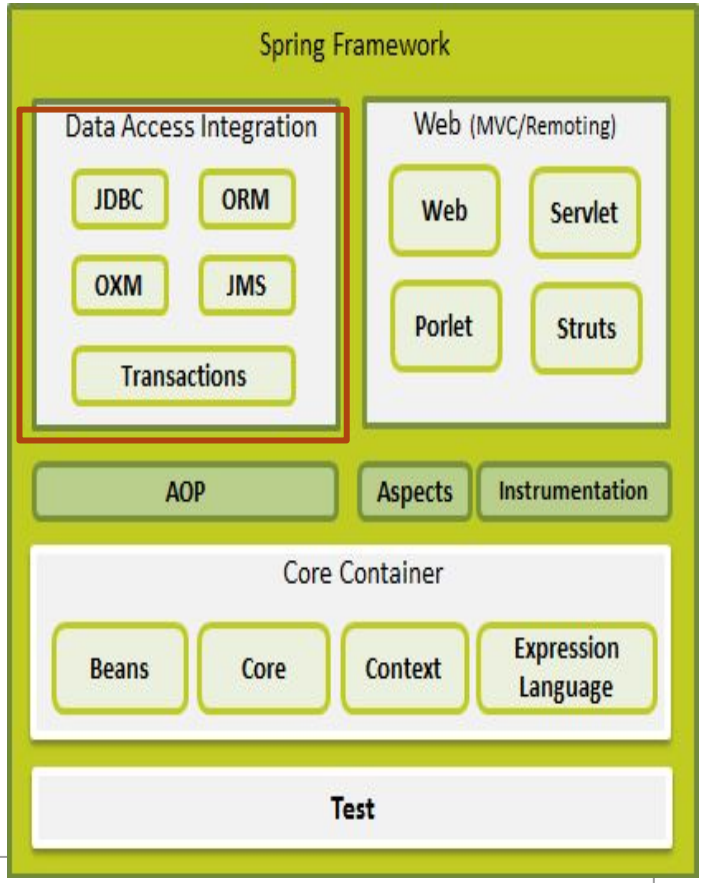
Struts is a Java-based web application framework used to build **MVC (Model–View–Controller)** applications for enterprise websites.

Spring Framework Architecture

AOP Layer-
This layer handles cross-cutting concerns like logging, security, and transactions using AOP.

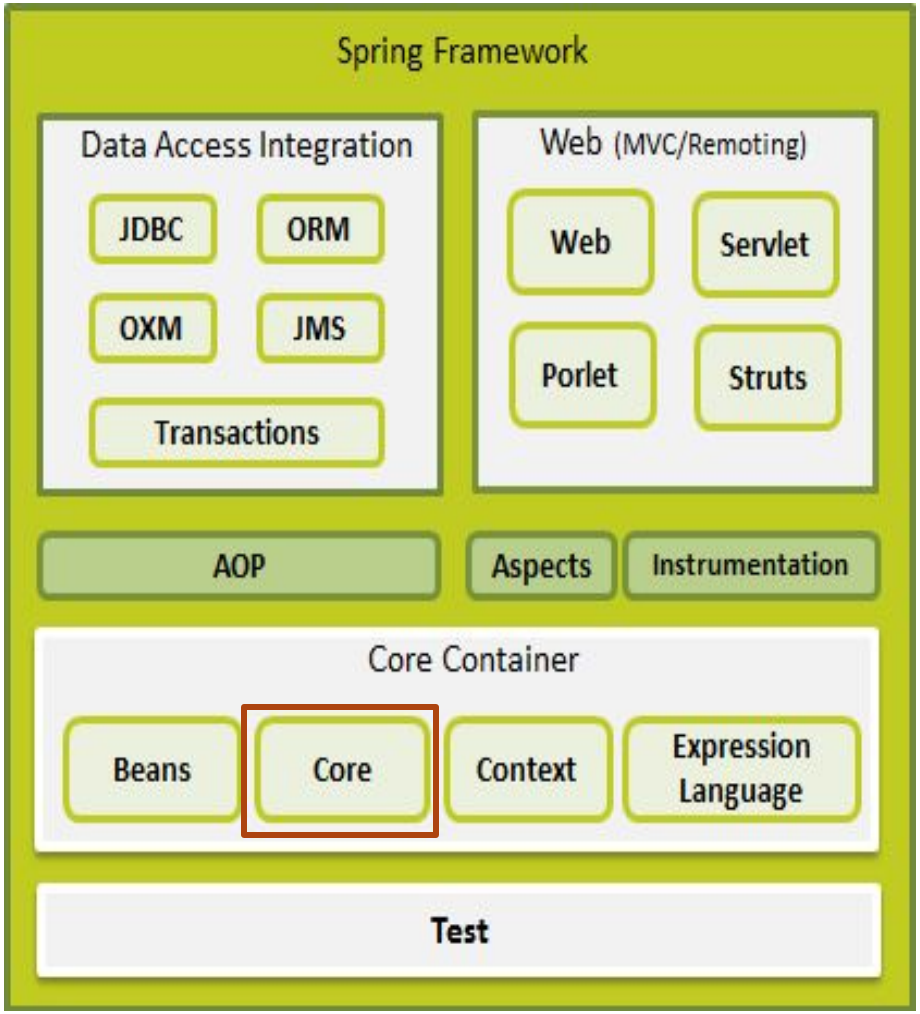
An Aspect is a module/class that contains cross-cutting concerns (like logging or security).

Instrumentation is the process of modifying or enhancing code behavior at runtime or load time.



Term	Meaning
AOP	Concept to separate common services
Aspect	Class that implements those services
Instrumentation	Technique to modify behavior dynamically

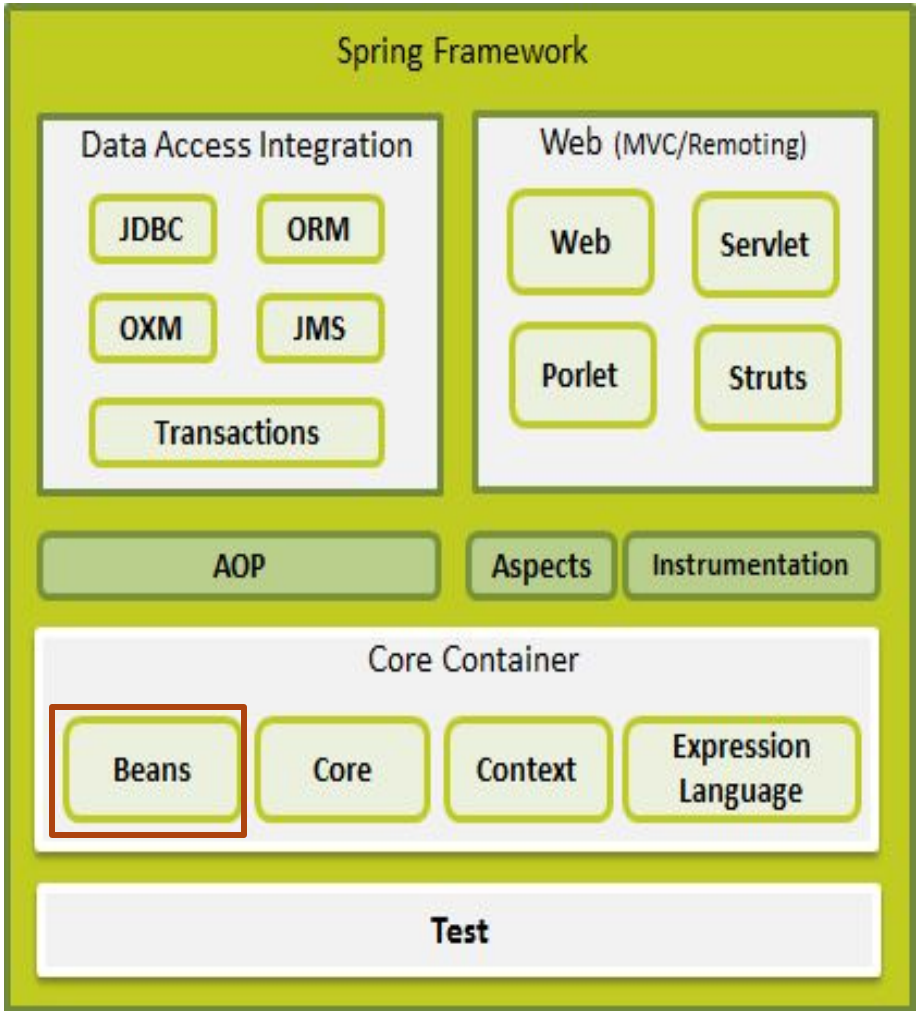
Spring Framework Architecture



Core Module :

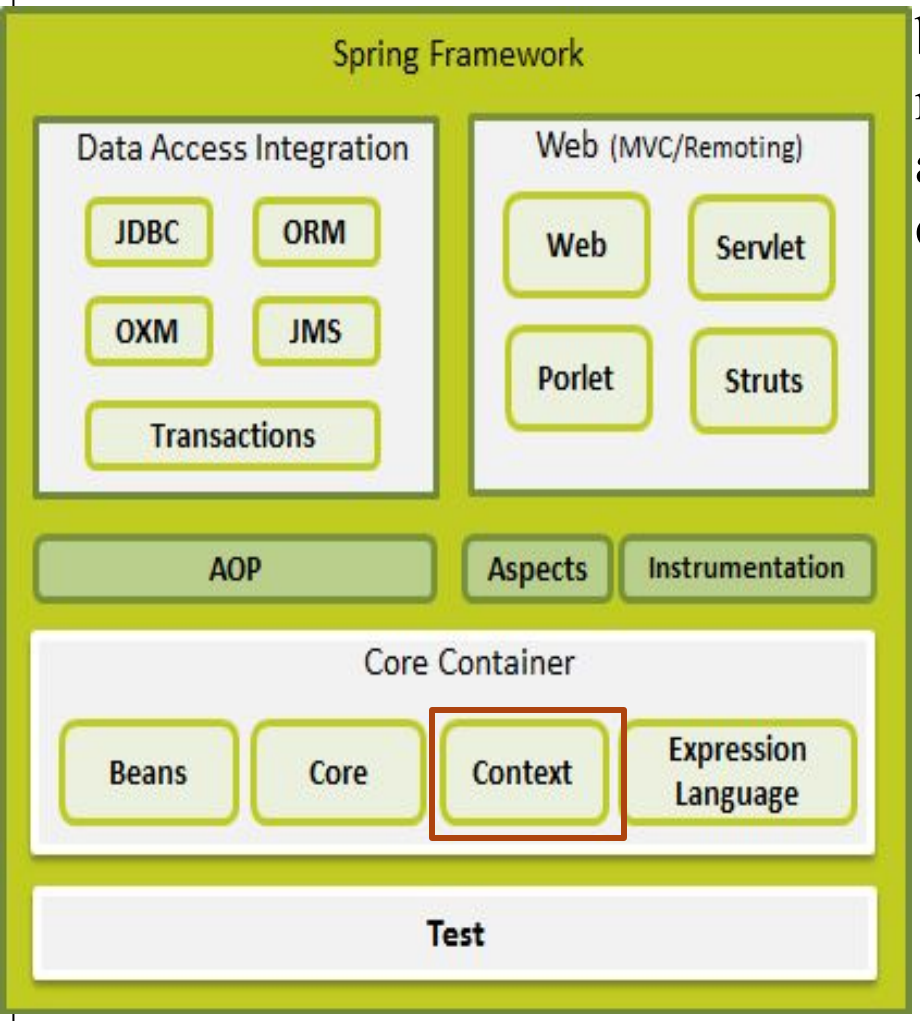
- The Spring container is responsible to create objects, wire them together and manage them from creation until destruction.
- The Spring container utilizes Dependency Injection to manage objects that make up an application.

Spring Framework Architecture



Beans Module provides BeanFactory,(preferred when the resources are limited such as mobile devices or applet based applications)

Spring Framework Architecture



Context Module builds on the solid base provided by the Core and Beans modules and it provides medium to access any objects defined and configured.

- ApplicationContext Container (Spring's more advanced container).This includes all functionality of BeanFactory. The most commonly used implementations are:
  - ***FileSystemXmlApplicationContext*** (loads definitions of the beans from an XML file. Need to provide full path of xml file)
  - ***ClassPathXmlApplicationContext*** loads definitions of the beans from an XML file. Does not need to provide the full path it will work with the xml file in the Classpath)
  - ***WebXmlApplicationContext***(loads the XML file with definitions of all beans from within a web application.)

A basic Spring Application

Create Maven Project

- Add Spring dependency in pom.xml
- <dependency>
- <groupId>org.springframework</groupId>
- <artifactId>spring-context</artifactId>
- <version>5.x.x</version>
- </dependency>

Create Processor Interface

```
public interface Processor {
    void process();
}
```

Intel Processor Class

```
public class IntelProcessor
implements Processor {
    public void process() {
        System.out.println("Intel
Processor");
    }
}
```

AMD Processor Class

```
public class AMDProcessor
implements Processor {
    public void process() {
        System.out.println("AMD
Processor");
    }
}
```

A basic Spring Application

Laptop Class

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class Laptop {
    private Processor processor;
    public void setProcessor(Processor processor) {
        this.processor = processor;
    }
    public void compile() {
        processor.process();
        System.out.println("Compiling...");
    }
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("beans.xml");
        Laptop laptop = (Laptop) context.getBean("laptop");
        laptop.compile();
    }
}
```

beans.xml

```
<beans
xmlns="http://www.springframework.org/schema/beans"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="

http://www.springframework.org/schema/beans

http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="intel" class="IntelProcessor"/>
    <bean id="amd" class="AMDProcessor"/>

    <bean id="laptop" class="Laptop">
        <property name="processor" ref="intel"/>
    </bean>

</beans>
```

Conclusion

- The most popular application development framework for enterprise Java
- Spring Framework (Architecture) is modular and allows you to pick and choose modules that are applicable to your application.
- POJO's (plain old java object) are called 'beans' and those objects instantiated, managed, created by Spring IoC container.
- The Spring IoC container makes use of Java POJO classes and configuration metadata to produce a fully configured and executable system or application.
- DI helps in gluing loosely coupled classes together and at the same time keeping them independent.

Introduction to Spring Boot

- Spring is one of the most popular frameworks for building enterprise applications, but traditional Spring projects require heavy XML configuration, making them complex for beginners.
- Spring Boot solves this problem by providing a ready-to-use, production-grade framework on top of Spring. It eliminates boilerplate configuration, comes with an embedded server and focuses on rapid development.

Traditional Spring Framework (Before Boot)

- XML-based configuration
- Manual dependency management
- Complex project setup
- External server required (e.g., Tomcat)

Example Problems:

- Too many XML files
- Complex setup for beginners
- Time-consuming configuration
- Difficult to start quickly

Challenges Faced by Developers

Challenge	Explanation
Configuration Overload	Large XML files
Dependency Conflicts	Version mismatch issues
Deployment Complexity	Need for external servers
Slow Development	More setup, less coding

Birth of Spring Boot

Spring Boot was introduced to:

- Simplify Spring application development
- Reduce boilerplate code
- Enable “just run” applications

Goal:

“Convention over configuration”

Features of Spring boot

- Auto-Configuration
- Embedded Server
- Opinionated Defaults

Auto-Configuration

Automatically configures application based on dependencies

- Reduces manual setup

How it Works:

- Detects libraries in classpath
- Applies default configuration

Example:

- Add spring-boot-starter-web

  Spring Boot configures:
  - DispatcherServlet
  - Tomcat server
  - Spring MVC

Embedded Server

- Built-in web server inside application

Supported Servers:

- Tomcat (default)
- Jetty
- Undertow

Without Spring Boot:

- Need to install and configure server manually

With Spring Boot:

✔ No external server required
✔ Run as a simple Java application

Application carries its own server.

Opinionated Defaults

- Predefined best configurations provided by Spring Boot

Meaning: Framework makes decisions for you

Examples:

- Default port → 8080
- Default server → Tomcat
- Default logging enabled

Customization:

- Defaults can be overridden

Evolution timeline

Phase 1: Spring Framework (Early Stage)

- XML-heavy
- Complex configuration
- Enterprise-level framework

Phase 2: Annotation-Based Spring

- Introduction of annotations:
  - @Component
  - @Autowired

Reduced XML usage

Phase 3: Java-Based Configuration

- Configuration using Java classes
- No XML required

Phase 4: Spring Boot (Modern Approach)

- Auto-configuration
- Embedded server
- Minimal setup
- Rapid development

Comparison: Spring vs Spring Boot

Feature	Spring Framework	Spring Boot
Configuration	XML / Java	Mostly automatic
Setup Time	High	Very low
Server	External	Embedded
Dependency Management	Manual	Starter-based
Ease of Use	Moderate	Easy

Role of Spring Boot in Modern Development

Spring Boot is widely used for:

- Web applications
- REST APIs
- Microservices

Spring Boot do not replaces Spring framework, rather it adds a layer on it to simplify java development.

Advantages and limitations of Spring Boot

Advantages

- ✔ Faster development
- ✔ Reduced configuration
- ✔ Easy deployment
- ✔ Microservices-friendly
- ✔ Beginner-friendly

Limitations

- Less control over configuration (in some cases)
- Can be heavy for very small applications

Spring Boot project structure

1. src/main/java

Contains **Java source code**

Sub-packages:

- **controller**

Handles HTTP requests (Web layer)

- **service**

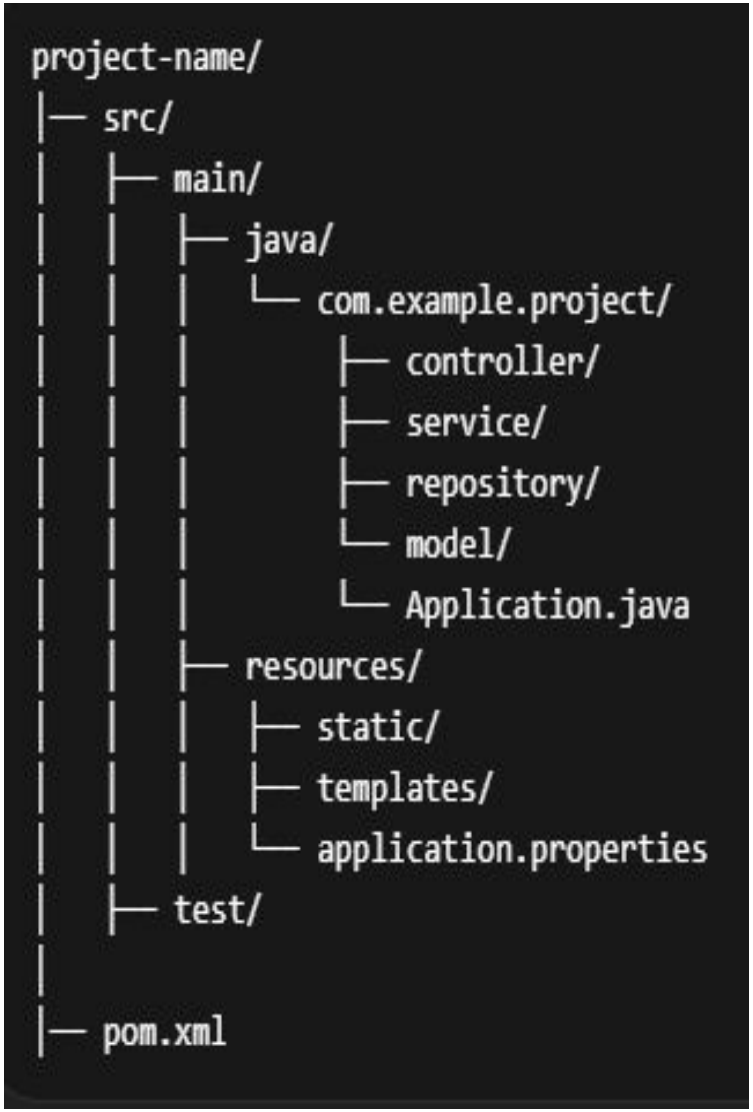
Contains business logic

- **repository**

Handles database operations

- **model (or entity)**

Defines data structure (POJO classes)



Spring Boot project structure

2. Main Class (Application.java)

Example

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Entry point of application
Starts embedded server
Triggers auto-configuration

Spring Boot project structure

3. src/main/resources

Stores configuration and static files

- **application.properties**

Configuration file

Examples:

server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/db

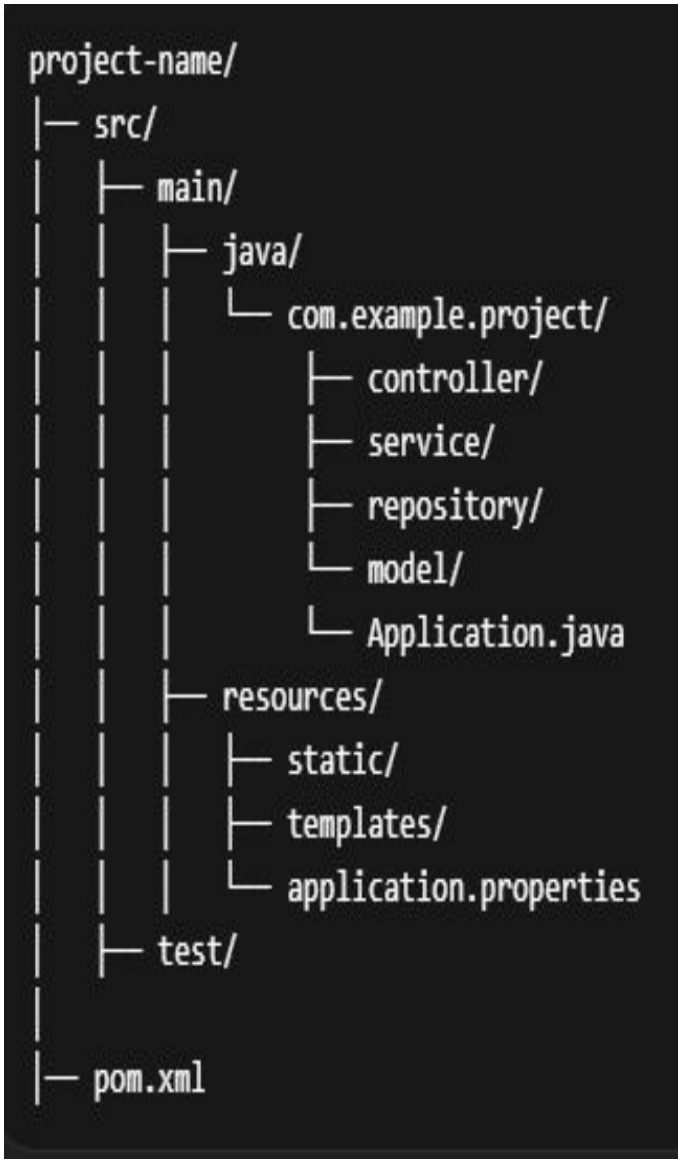
- **static/**

Contains static files:

- HTML
- CSS
- JS

- **templates/**

Used for dynamic pages (Thymeleaf, JSP)



Spring Boot project structure

4. src/test/java

Used for testing

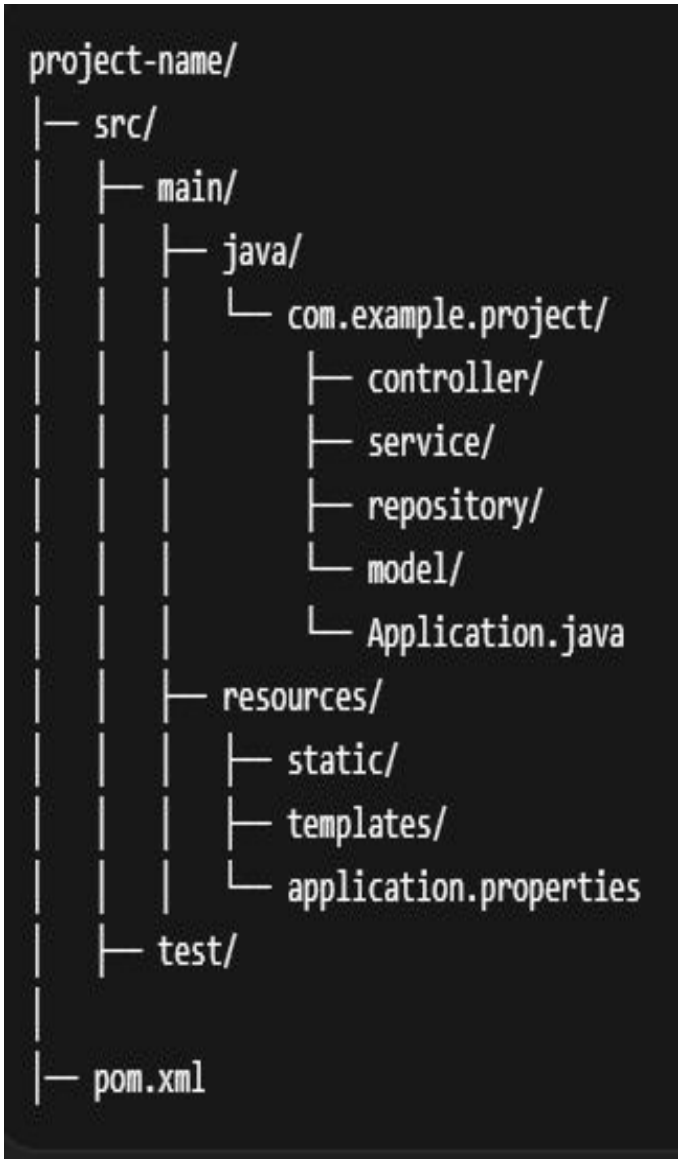
- Unit tests
- Integration tests

5. pom.xml

Maven configuration file

Contains:

- Dependencies (Spring Boot starter)
- Build configuration



A basic Spring Boot application

Create Project in STS

Fill details:

- Name: `SpringbootDemo`
- Type: Maven
- Packaging: Jar

Click **Next-Finish**

Right click on project name - create new class-Device

.

```
package com.example.demo;
import
org.springframework.stereotype.C
omponent;
@Component
public class Device {
void deviceDetails()
{
    System.out.println("Device
Name:Lenovo Laptop");
}
}
```

Create Student Class

```
package com.example.demo;
import
org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
@Component
public class Student {
private int Sid;
private String name;
@Autowired
Device d;
void show()
{
    System.out.println("Student enrolled...");
    d.deviceDetails();
}}
```

`@Component` → tells Spring to create object (bean)
`@Autowired` → injects dependency automatically

Main Class-StudentAppApplication

```
package com.example.demo;
import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ConfigurableApplicationContext;
@SpringBootApplication
public class StudentAppApplication {
    public static void main(String[] args) {
        ConfigurableApplicationContext
context=SpringApplication.run(StudentAppApplication.class, args);
        Student s=context.getBean(Student.class);
        s.show();
        System.out.println("Welcome to Spring Boot");
    }
}
```

Right-click → **Run As** → **Spring Boot App**

Output

Student enrolled...

Device Name:Lenovo Laptop

Welcome to Spring Boot

1. Spring starts application
2. Scans package for `@Component`
3. Creates objects (beans)
4. Finds `@Autowired`
5. Injects `Device` into `Student`

Concept Mapping

Concept	Used Here
IoC	Spring creates objects
DI	`@Autowired`
Bean	`@Component`
Container	ApplicationContext

Creating a web application

Create Project in STS

File → New → Spring Starter Project

Fill Details:

- Name: `WebDemo`
- Type: Maven
- Packaging: Jar
- Java Version: 8+

Click **Next**

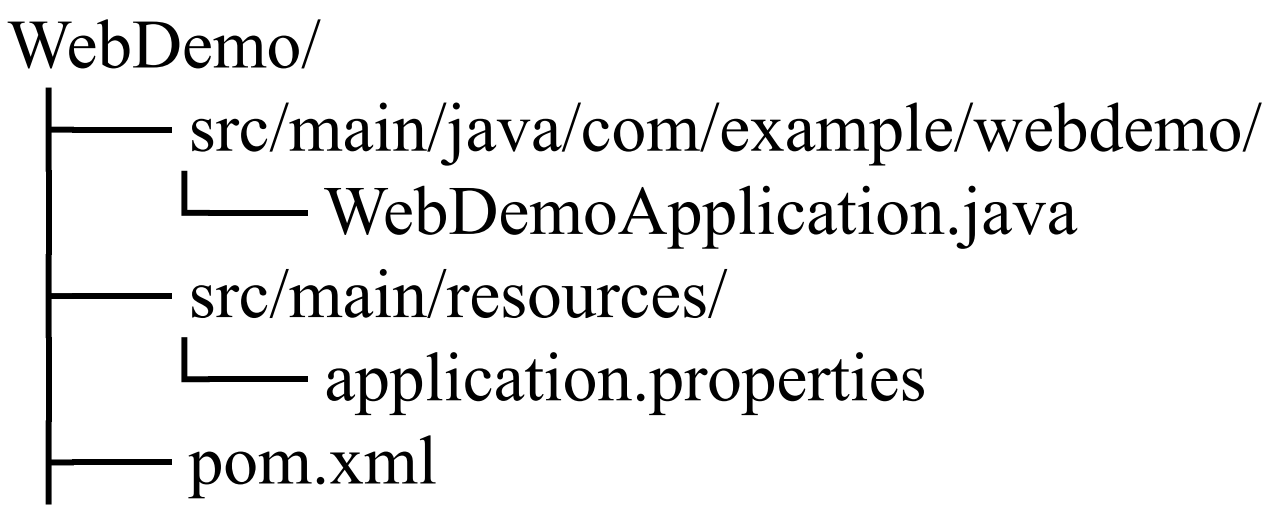
Add Dependency

Select:

- **Spring Web**

Click **Finish**

Project Structure



Main Class-WebDemoApplication.java

```
package com.example.webdemo;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class WebDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(WebDemoApplication.class, args);
    }
}
```

- `@SpringBootApplication` → enables Spring Boot features
- Starts embedded server (Tomcat)

Create Controller

Create package: `controller`

Create class: `HomeController.java`

-

```
package com.example.webdemo.controller;
import org.springframework.web.bind.annotation.GetMapping;
import
org.springframework.web.bind.annotation.RestController;

@RestController
public class HomeController {

    @GetMapping("/")
    public String home() {
        return "Welcome to Spring Boot Web Application";
    }
}
```

`@RestController` → handles web requests
`@GetMapping("/")` → maps URL

Right-click project →
Run As → Spring Boot App

Test in Browser

http://localhost:8080/

Output:

Welcome to Spring Boot Web Application

Understand Flow

Browser → Request → Controller → Response → Browser

Spring Security

Authentication & Authorization

- The authorize & authentication tag

```
<sec:authorize access="hasRole('supervisor')">

This content will only be visible to users who have
the "supervisor" authority in their list of <tt>GrantedAuthority</tt>s.

</sec:authorize>
```

```
<sec:authorize access="isAuthenticated()">
    <img src="https://s3.amazonaws.com/serhat/${spt.id}.jpg" align="middle" />
    <span><sec:authentication property="principal.username" /></span>
    <br />
    <s:url value="/static/j_spring_security_logout" var="logout_url" />
    <a href="${logout_url}">Logout</a>
    <sec:authorize url="/admin">
        <s:url value="/admin" var="admin_url" />
        <br />
        <a href="${admin_url}">Admin</a>
    </sec:authorize>
</sec:authorize>
```

Spring Security

Authentication & Authorization

- You can access the Authentication object in your MVC controller (by calling SecurityContextHolder.getContext().getAuthentication()) and add the data directly to your model for rendering by the view.

```
Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
String currentPrincipalName = authentication.getName();
```

```
@Controller
public class SecurityController {

    @RequestMapping(value = "/username", method = RequestMethod.GET)
    @ResponseBody
    public String currentUserName(Authentication authentication) {
        return authentication.getName();
    }
}
```

Spring Security

Authentication & Authorization

- Authorization with annotations in RESTful Web Service

```
@PreAuthorize("hasRole('ROLE_ADMIN')")
@RequestMapping(value = "/delete/{id}", method = RequestMethod.DELETE, produces = "application/json")
public boolean deleteCustomer(@PathVariable Long id) {
    return customerService.deleteCustomer(id);
}
```

```
@Autowired
private CustomerService customerService;

@PreAuthorize("isAuthenticated()")
@RequestMapping(value = "/customers/retrieve", method = RequestMethod.GET, produces = "application/json")
public List<Customer> getCustomers() {
    return customerService.getCustomers();
}
```

References

- http://www.cs.colorado.edu/~kena/classes/5448/f11/ lectures/30-dependencyinjection.pdf
- Spring Framework 3.1 Tutorial
- http://courses.coreservlets.com/Course-Materials/ spring.html
- http://martinfowler.com/articles/injection.html